

Lab Sheet 06

Registration No. : 2025/CS/079

Index No. : 25000799

Temp. Index No. : 25001078

A.

1. Basic operation: print(i, j)

When $i = 0$ basic operation occur for 0 times

When $i = 1$ basic operation occur for 1 times

When $i = 2$ basic operation occur for 2 times

When $i = n$ basic operation occur for n times

$$\begin{aligned} 1 + 2 + 3 + \dots + n &= n(n + 1)/2 \\ &= 1/2n^2 + 1/2n \end{aligned}$$

Time complexity = $O(n^2)$

2. Basic operation: print(i, j, k)

k repeats p times. $\rightarrow p$

j repeats k again for m times $\rightarrow m \times p$

i repeats j again for n times $\rightarrow n \times m \times p$

Time complexity = $O(nmp)$

3. Basic Operation: print(i, j, k)

When $i = 0$, j occur for 0 times

When $i = 1$, j occur for 1 times

When $i = 2$, j occur for 2 times

When $i = n$, j occur for n times

$$\begin{aligned} 1 + 2 + 3 + \dots + n &= n(n + 1)/2 \\ &= 1/2n^2 + 1/2n \end{aligned}$$

When the condition satisfies: n^2

When $j = 0$ basic operation occur for 0 times

When $j = 1$ basic operation occur for 1 times

When $j = 2$ basic operation occur for 2 times

When $j = n$ basic operation occur for n times

$$\begin{aligned} 1 + 2 + 3 + \dots + n &= n(n + 1)/2 \\ &= 1/2n^2 + 1/2n \end{aligned}$$

When the condition does not satisfy: $n^2 \times n$ (since j is dependent of i)

$$n^3 + n^2$$

Time complexity = $O(n^3)$

4. Basic operation: print(i, j)

When $i = 1$ basic operation occur for 1 times

When $i = 2$ basic operation occur for 2 times

When $i = 4$ basic operation occur for 4 times

When $i = n - 1$ basic operation occur for $n - 1$ times

$$1 + 2 + 4 + 8 + \dots + n = 2n - 1$$

Time complexity = $O(n)$

5. Basic operation: print(i, j, k)

When $i = 0$, j occur for 0 times

When $i = 1$, j occur for 1 times

When $i = 2$, j occur for 2 times

When $i = n$, j occur for n times

$$\begin{aligned} 1 + 2 + 3 + \dots + n &= n(n + 1)/2 \\ &= 1/2n^2 + 1/2n \end{aligned}$$

When the condition does not satisfy: n^2

When $j = 0$ basic operation occur for 0 times

When $j = 1$ basic operation occur for 1 times

When $j = 2$ basic operation occur for 2 times

When $j = n$ basic operation occur for n times

$$\begin{aligned} 1 + 2 + 3 + \dots + n &= n(n + 1)/2 \\ &= 1/2n^2 + 1/2n \end{aligned}$$

When the condition does not satisfy: $n^2 \times n$ (since j is dependent of i)

$$n^3 + n^2$$

Time complexity = $O(n^3)$

6. Basic operation: print(i, k)

When $i = 0$, k goes to n by $i + 1$ increments starting from 0.

When $i = 1$, k goes to n by $i + 1$ increments starting from 1.

When $i = n - 1$, k goes to n by $i + 1$ increments starting from $n - 1$.

The while loops time complexity = $O(\log n)$

The for loop time complexity = $O(n)$

Time complexity = $O(n \log n)$

B.

```
1. L = []
   n = length(L)
   max = L[0]
   for i from 0 to n - 1:
       if L[i] > max:
           max = L[i]
```

Basic Operation: max = L[i]

Loop runs n times to determine the maximum,

Time complexity: O(n)

```
2. L = []
   n = length(L)
   min = L[0]
   for i from 0 to n - 1:
       if L[i] < min:
           min = L[i]
```

Basic Operation: min = L[i]

Loop runs n times to determine the minimum,

Time complexity: O(n)

```
3. L = []
   n = length(L)

   for i from 0 to n - 1:
       for j from i to n - 1:
           for k from j to n - 1:
               print(L[i], L[j], L[k])
```

Basic Operation: print(L[i], L[j], L[k])

When i = 0, j occur for 0 times

When j = 0 basic operation occur for 0 times

When i = 1, j occur for 1 times

When j = 1 basic operation occur for 1 times

When i = 2, j occur for 2 times

When j = 2 basic operation occur for 2 times

When i = n, j occur for n times

When j = n basic operation occur for n times

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$$

$n^2 \times n$ (since j in dependent of i)

Time complexity = O(n³)

C.

(a) $T(n) = n$

For new computer,

$$T(n) = n \times 1000$$

$$T(1000) = 1000 \times 1000$$

$$= 10^6$$

$$T(n) = 10^6$$

$$n = 10^6$$

(b) $T(n) = n^3$

For new computer,

$$T(n) = n^3 \times 1000$$

$$T(1000) = 1000^3 \times 1000$$

$$= 10^9 \times 10^3$$

$$= 10^{12}$$

$$T(n) = 10^{12}$$

$$n^3 = 10^{12}$$

$$n = 10^4$$

(c) $T(n) = 10^n$

For new computer,

$$T(n) = 10^n \times 1000$$

$$T(1000) = 10^{1000} \times 1000$$

$$= 10^{1000} \times 10^3$$

$$= 10^{1003}$$

$$T(n) = 10^{1003}$$

$$10^n = 10^{1003}$$

$$n = 1003$$

D. Growth rate in expression,

$$3n^2 > 10n \log n > 1000n > 4 \log n > 9999$$

therefore the growth rate is more dependent on $3n^2$,

therefore the complexity category is $\theta(n^2)$

(e)

E.

1. $25 + 32 = 55$
output is 55

2. The first loop runs n times; $T(n) = n$
The second loop runs n times; $T(n) = n$

$$T(n) = n + n = n$$

3.

```
int add_them (int n, int A[])
{
    index i, j, k;
    j = 0;
    for (i = 1; i <= n; i++)
        j = j + A[i];
    k = 2n;
    return j + k;
}
```

The loop runs n times; $T(n) = n$

$$T(n) = n$$